Network Mesh Implementation with Cilium

Cilium container networking and power of eBPF technology

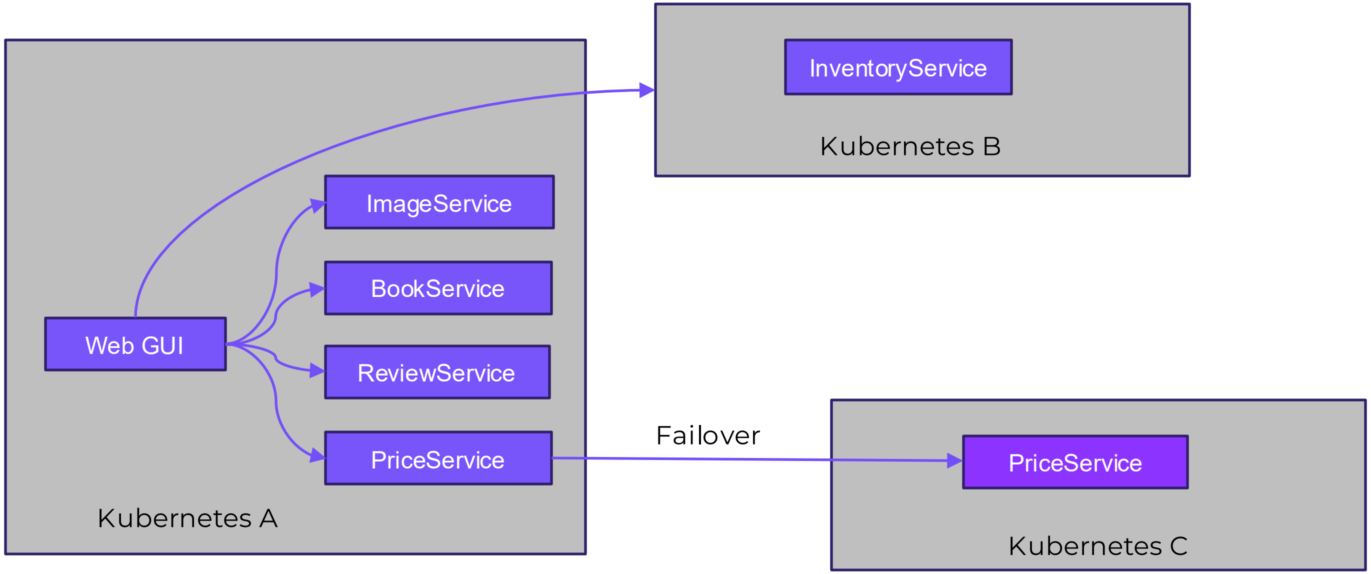
- Pongsak Khamdee (Tao)
- Senior Systems Engineer
- Nutanix Thailand
- pongsak.khamdee@nutanix.com
    - Life: https://pongsak.online
    - Ex VMware
    - Ex Pivotal
    - Ex Redhat Consult, Marlo Group
    - Ex True, TrueMove, Orange

CERTIFIED kubernetes APPLICATION DEVELOPER

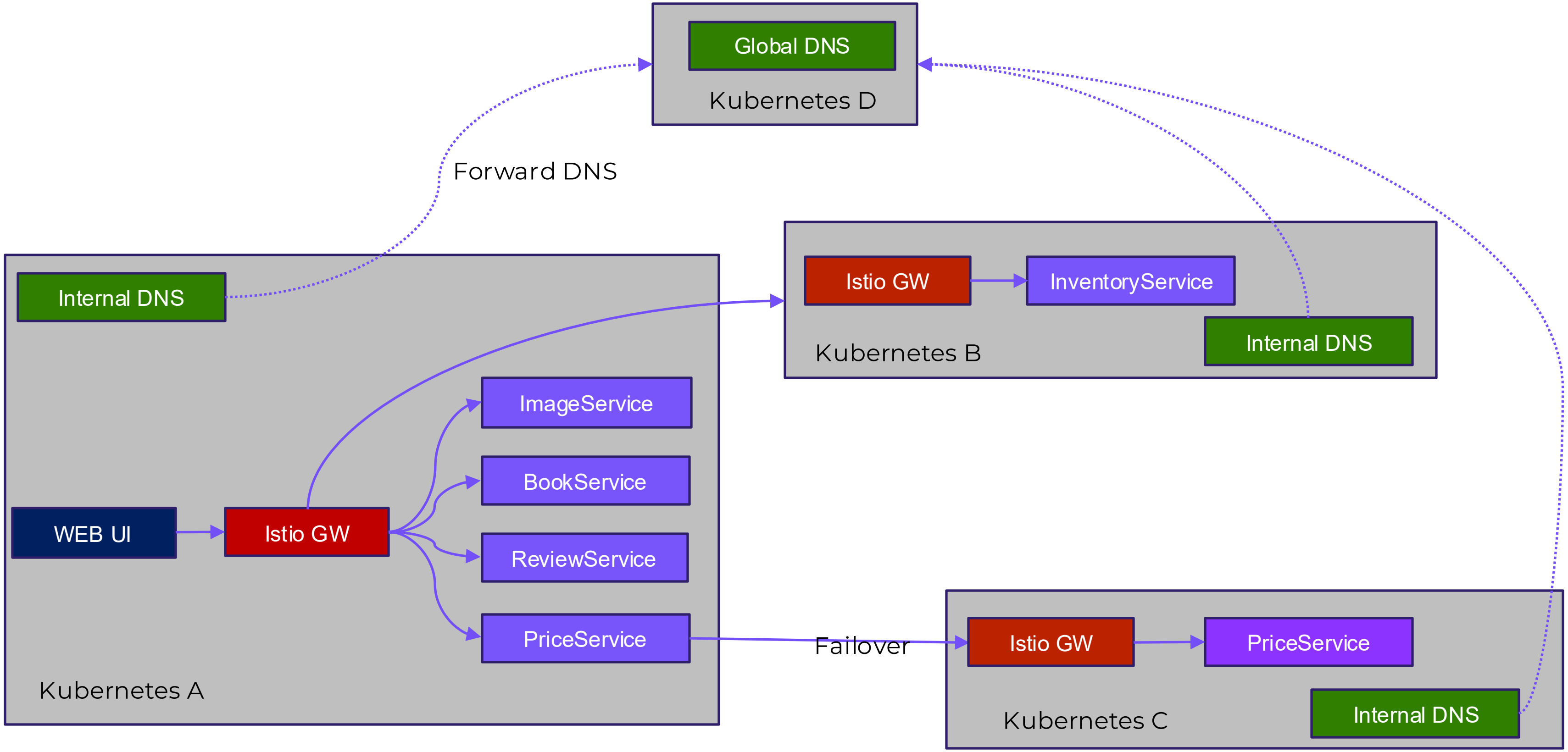
CERTIFIED kubernetes ADMINISTRATOR

amazon.co.uk
Hello. Sign i
Your Amaz
Page Context:
{ type: Book, id: ISBN-10 0-321-83457-7 }
Shop All Departments
Search
Books
Advanced Search
Browse Genres
Bestsellers
New & Future Releases
Paperback
Domain-Driven Design and c
ooks are available for Am
ImageService
BookService
Click to LOOK INSIDE!
Domain-Driven DESIGN
Tackling Complexity in the Heart of Software
Domain-driven Design: Tackling Complexity in the Heart of Software [Hardcover]
Eric Evans (Author)
(12 customer reviews)
Like (15)
ReviewService
RRP: £46.99
Price: £30.09 & this it
in the UK with Super Saver Delivery. See details and
You Save: £16.90 (36%)
PriceService
In stock.
InventoryService
Dispatched from and sold by Amazon.co.uk. Gift-wrap available.
Want guaranteed delivery by Friday, March 23? Order it in the next 21 hours and 9 minutes, and choose Express de
Customers Who Bought This Item Also Bought
OthersAlsoBoughtService
ImageService
BookService
Patterns of Enterprise Application Architecture... by Martin Fowler
(14)
£33.59
Growing Object-Oriented Software, Guided by Test... by
£18.35
Clean Code: A Handbook of Agile Software Craf... by Robert C. Martin
(27)
£17.00
Enterprise Integration Patterns: Designing, Build... by Gregor Hohpe
(16)
£29.61
Continuous Delivery: Reliable Software Releases Thr... by Jez Humble
(7)
£19.97
Refactoring: Improving the Design of Existing Co... by Martin Fowler
(24)
£29.61
ReviewService
PriceService
Product details
Hardcover: 560 pages
Publisher: Addison Wesley; 1 edition (20 Aug 2003)
BookService

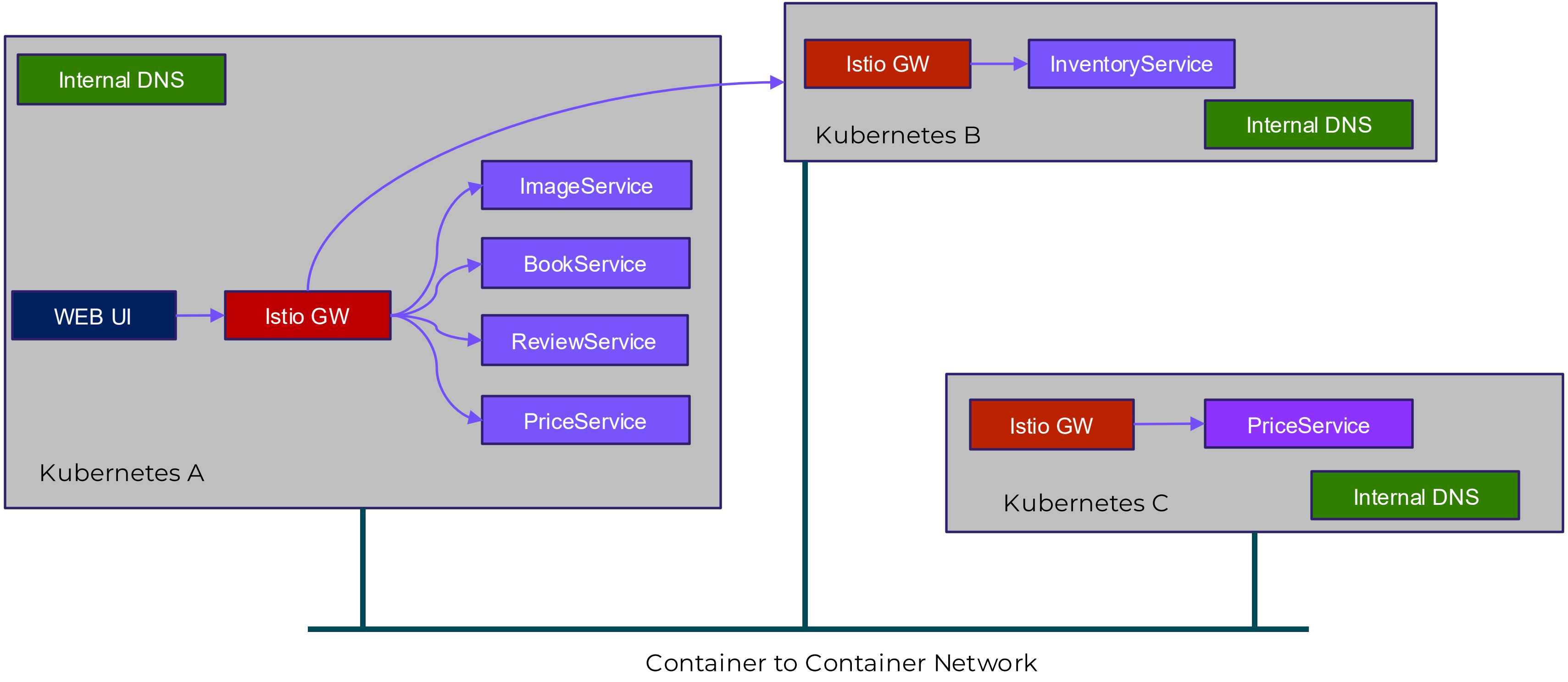
Multi-Cluster deployment



Multi-Cluster deployment



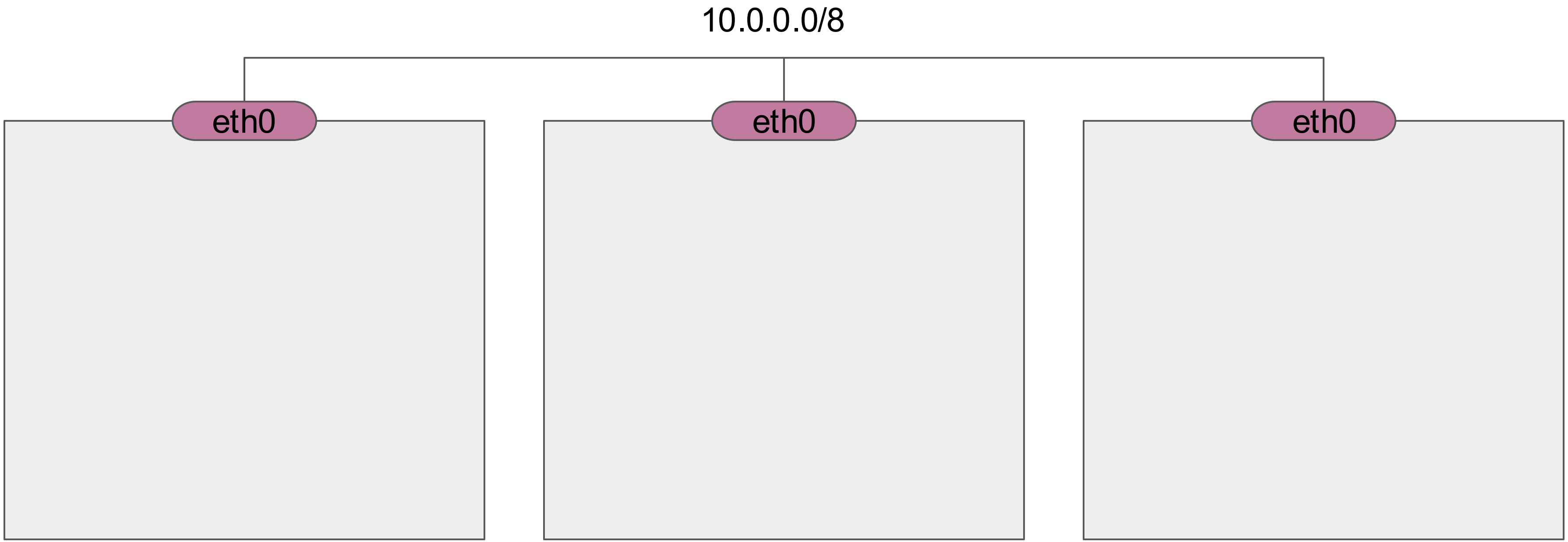
Cluster Mesh



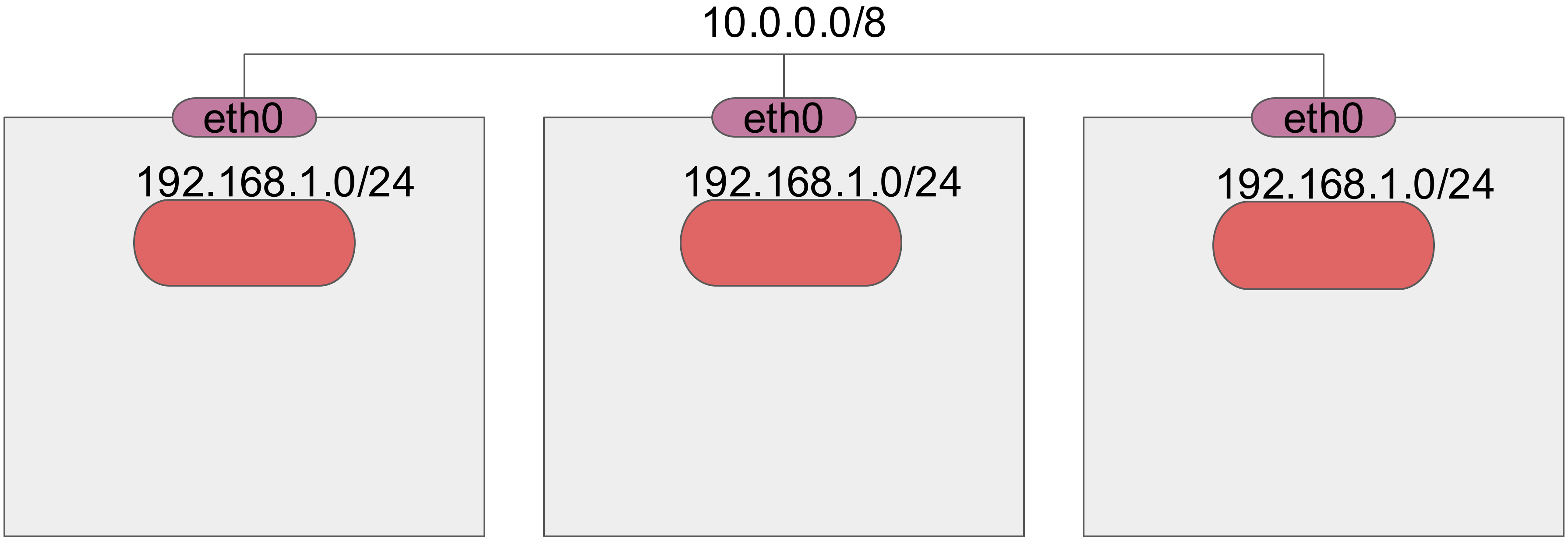
Kubernetes fundamental network requirement

- **Pod-to-Pod Communication without NAT:** Every pod in a Kubernetes cluster receives a unique IP address. The network must allow direct communication between any two pods, regardless of whether they reside on the same node or different nodes, without requiring Network Address Translation (NAT) for internal pod-to-pod traffic.
- **Node-to-Pod Communication:** Agents running on a node, such as system daemons and the Kubelet, must be able to communicate with all pods running on that specific node.
- **Service-to-Pod Communication:** Kubernetes Services provide a stable abstraction layer for accessing pods. The network must enable communication from Services to the underlying pods they expose.
- **External Connectivity (Ingress/Egress):** For applications to interact with the outside world, the network needs to facilitate both ingress (incoming) and egress (outgoing) traffic. This often involves using NAT gateways for pods to access external resources and mechanisms like Ingress controllers or Load Balancers to expose Services to external clients.

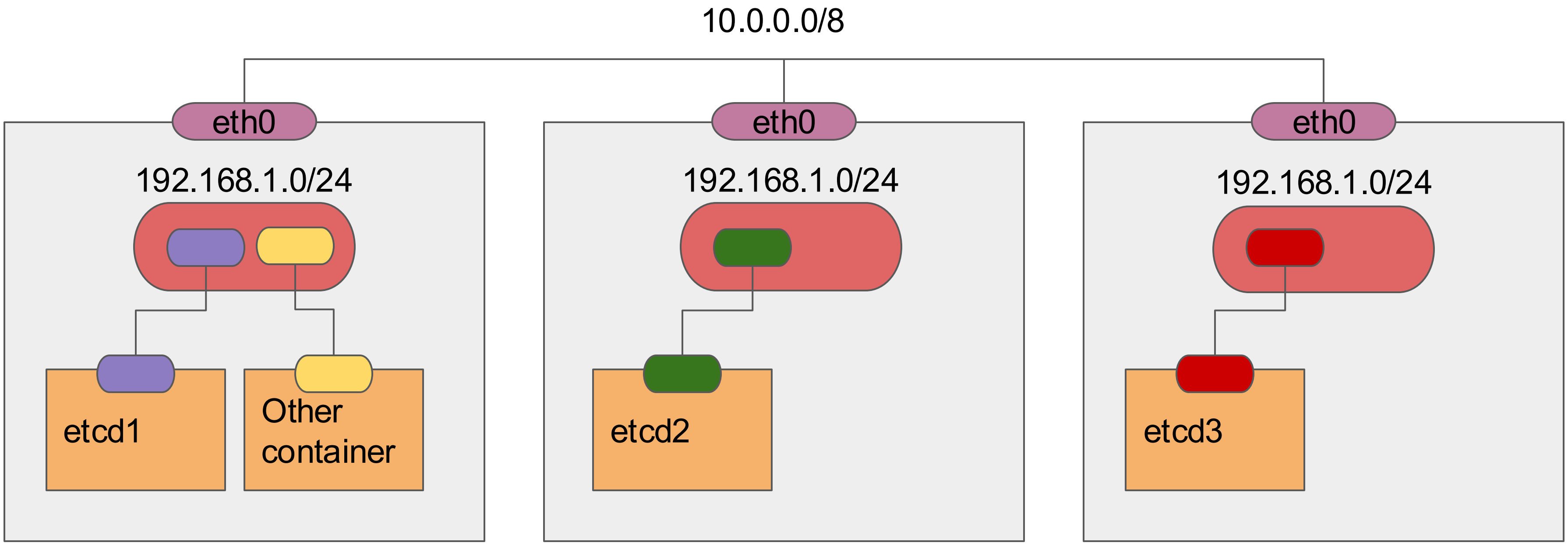
Let’s get 3 VMs



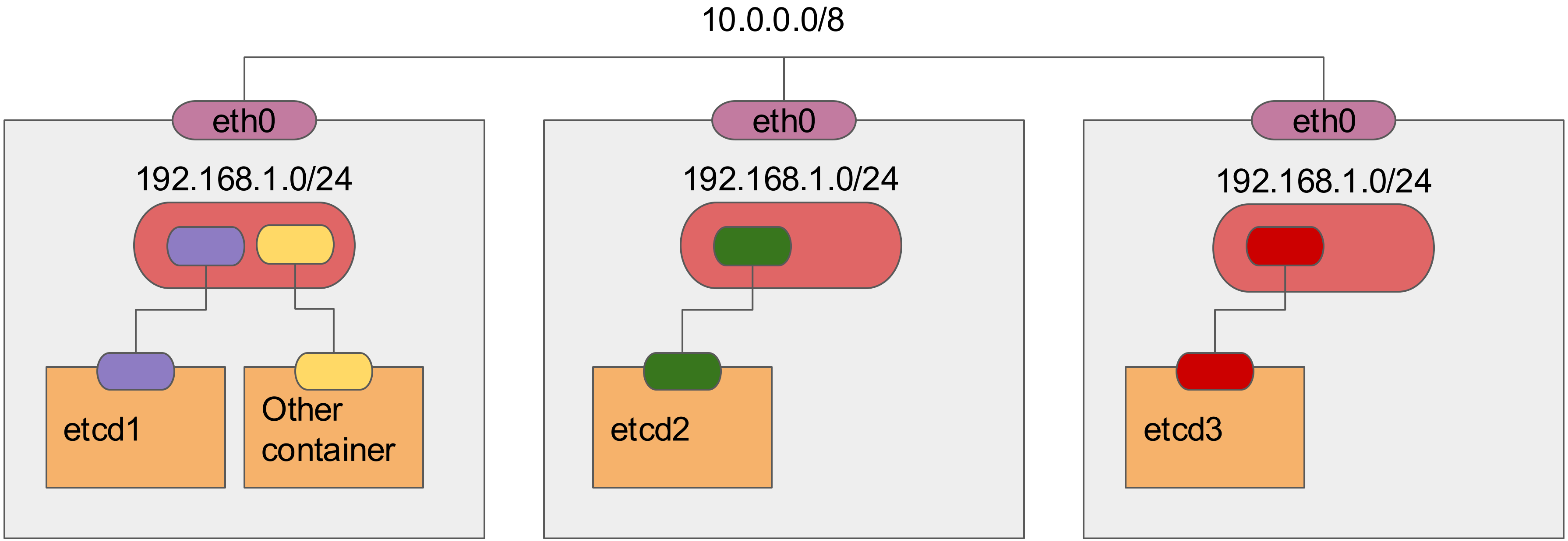
Create a bridge on each of them



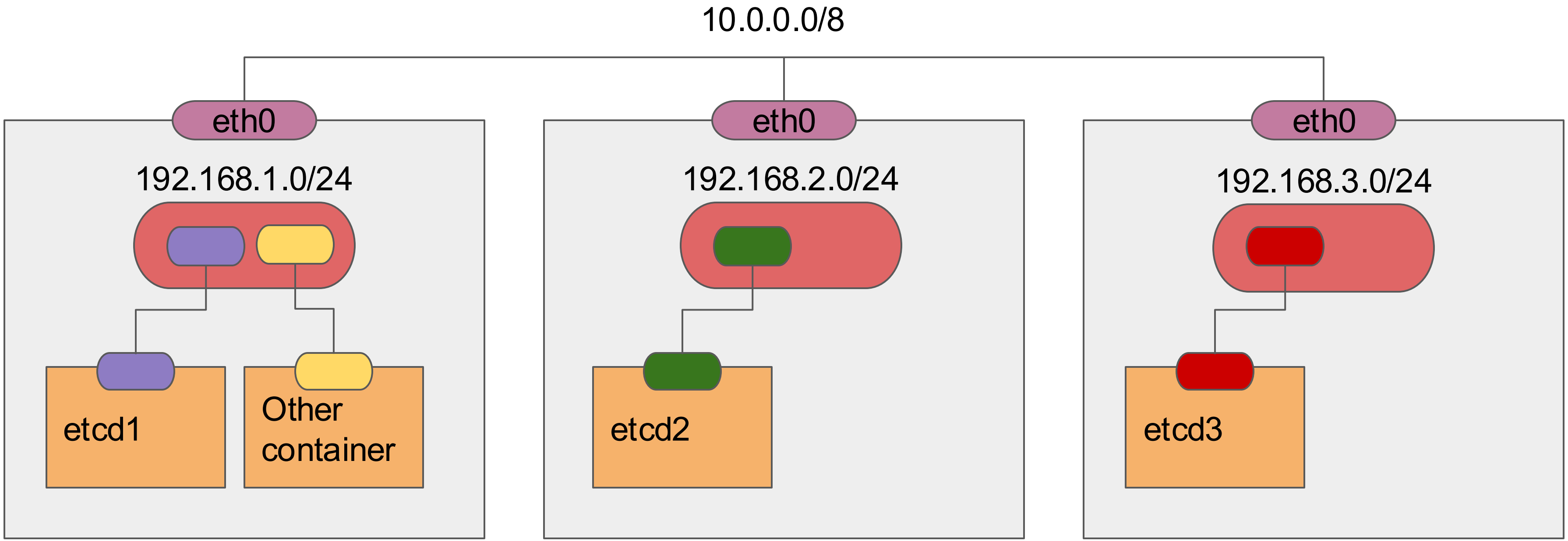
And start the containers



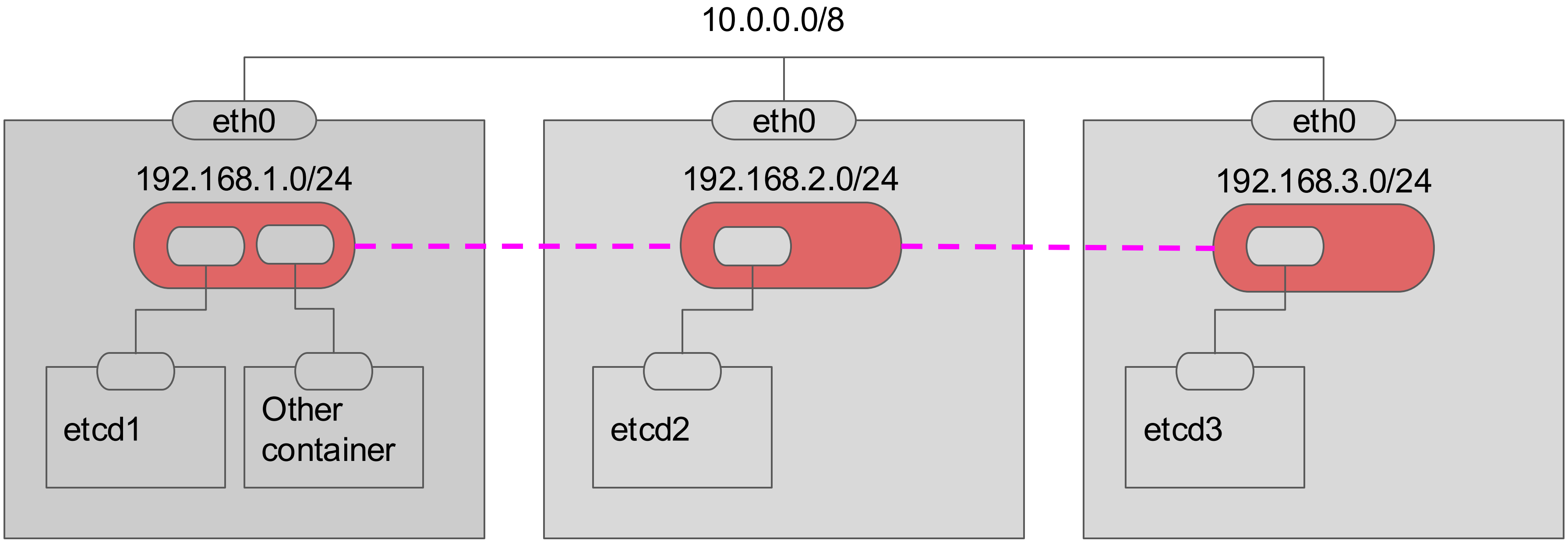
Wait
If all the bridges have same subnet, Things can get messy



So, let's give them a subnet each



But, how does a packet get routed from etcd1 to etcd2?



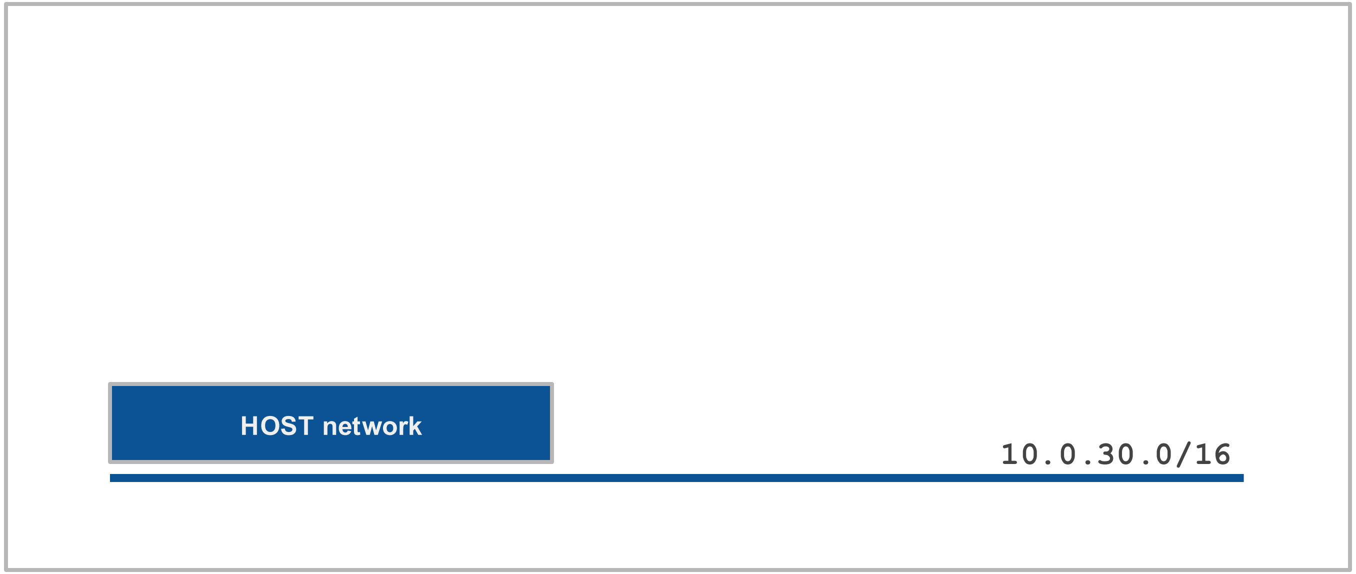
There are many ways

So, some smart people got together and wrote a specification

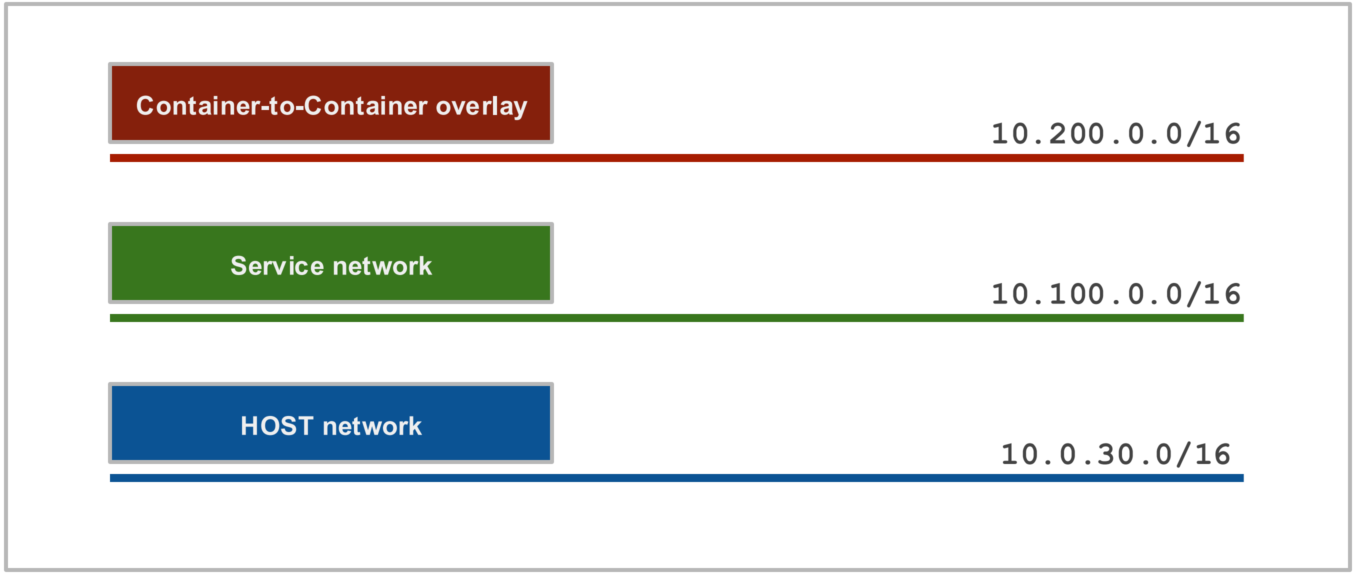
Which is called
CNI
(Container Networking Interface)

Interface between Container Orchestration Engines and Container Runtimes

Networking

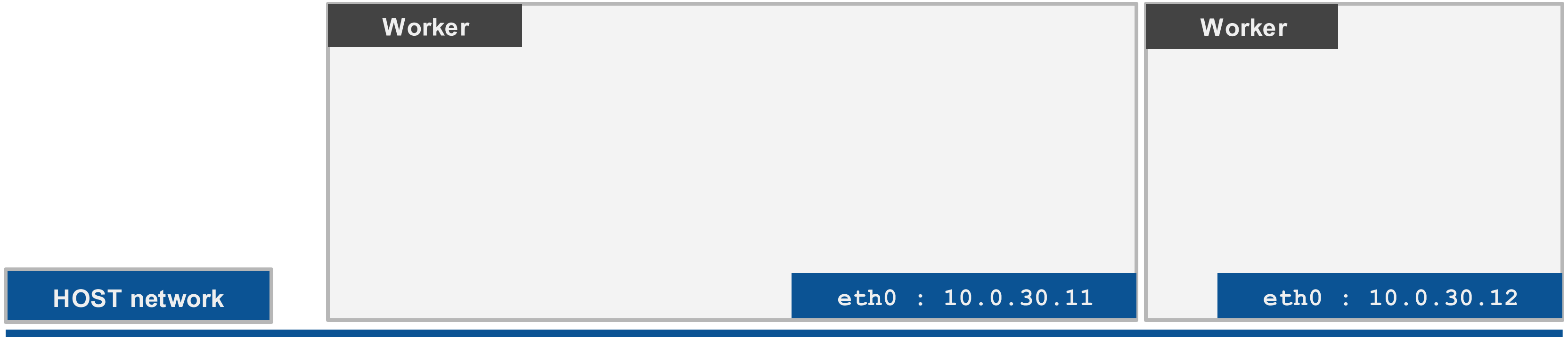


Networking

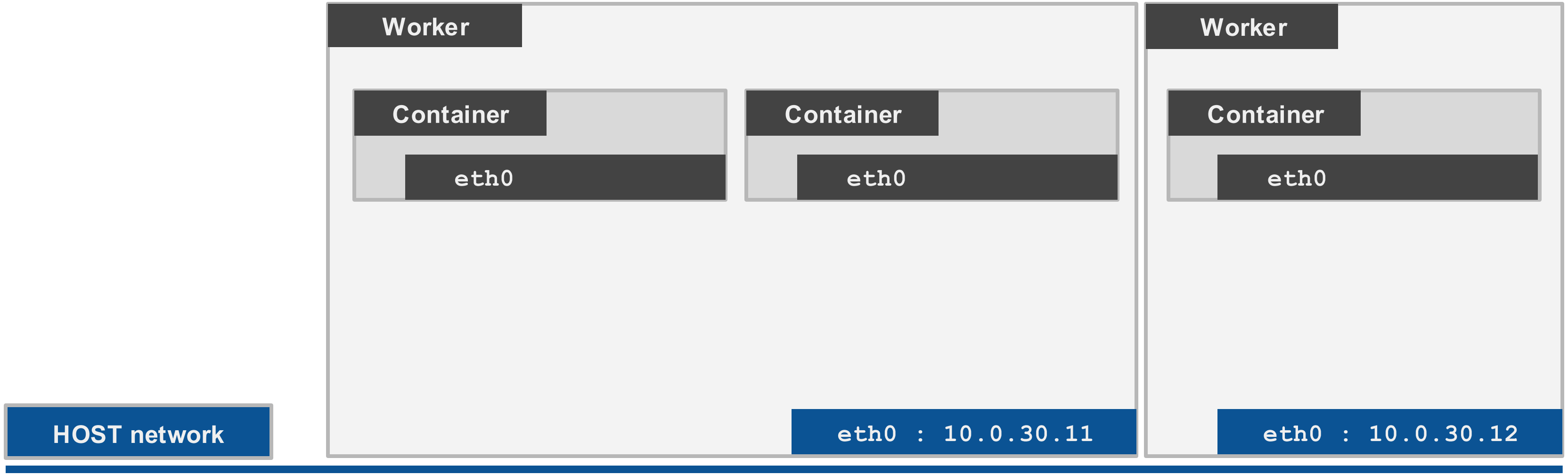


Container-to-Container overlay
10.200.0.0/16
Service network
10.100.0.0/16
HOST network
10.0.30.0/16

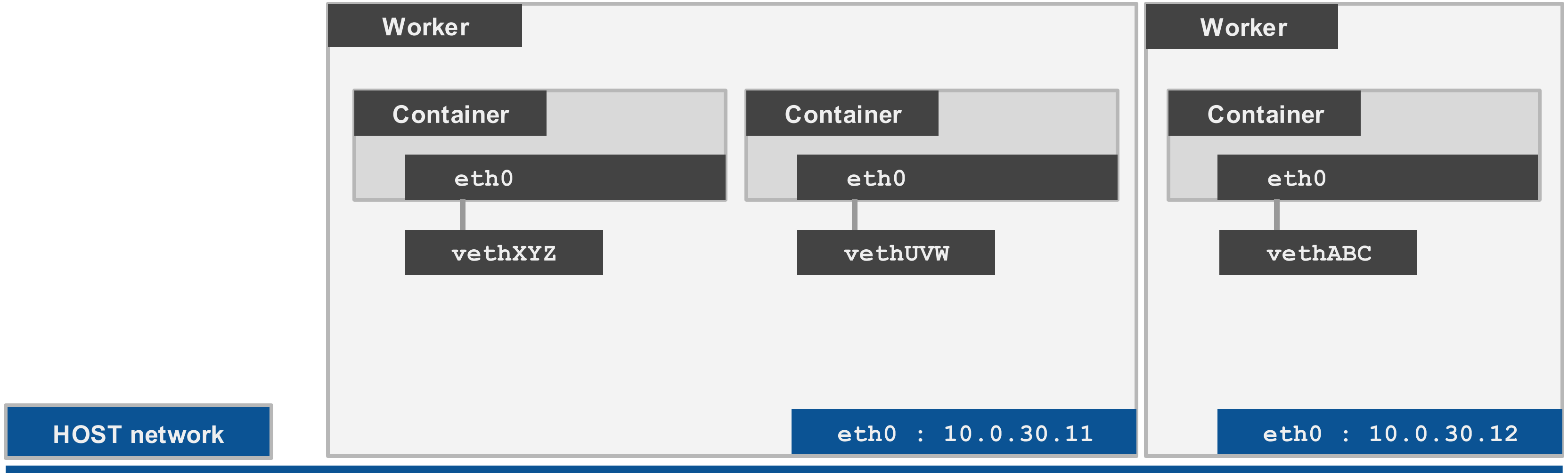
Networking



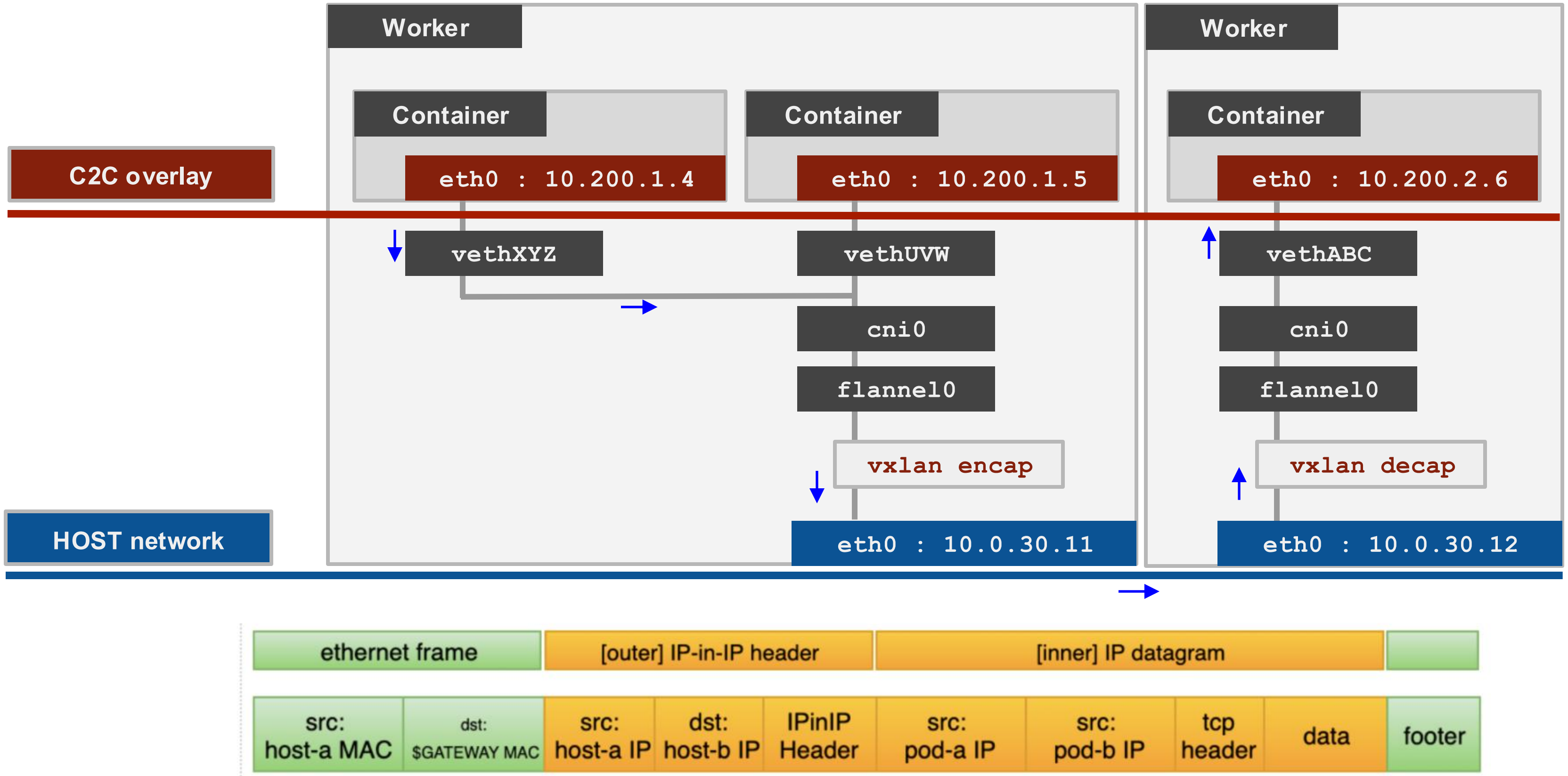
Networking


Worker
Container
eth0
Container
eth0
eth0 : 10.0.30.11
Worker
Container
eth0
eth0 : 10.0.30.12
HOST network

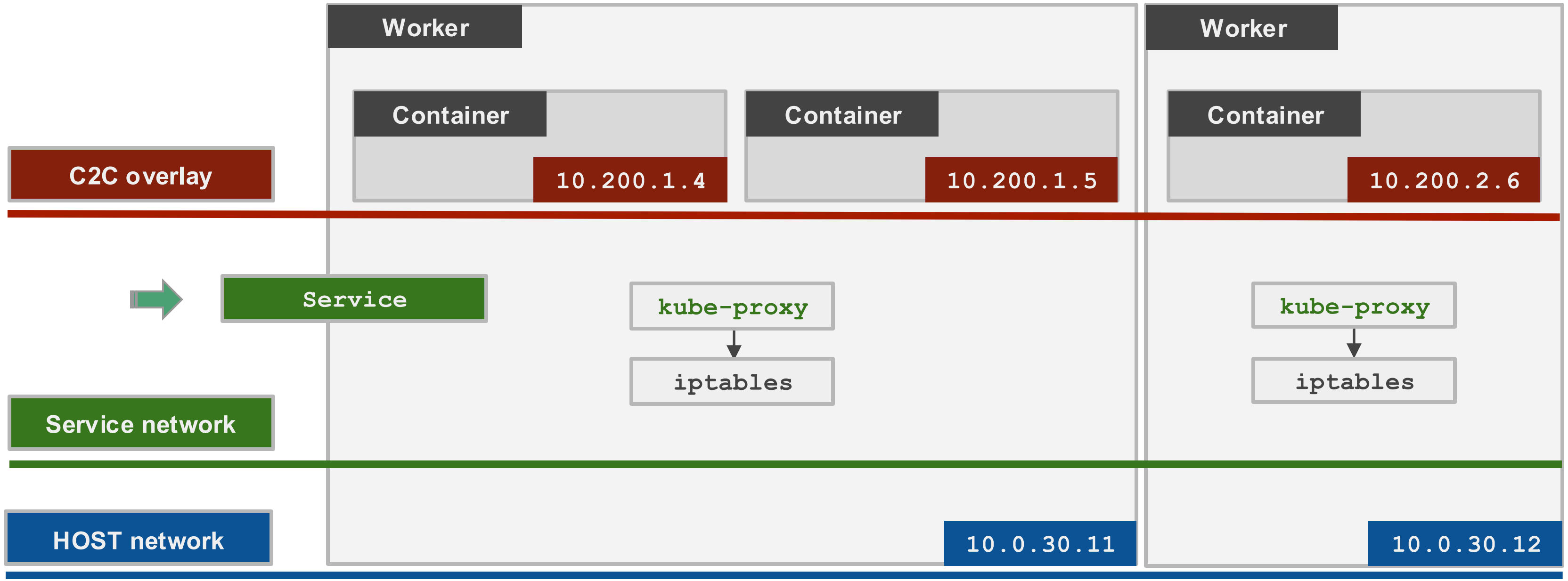
Networking


Worker
Container
eth0
vethXYZ
Container
eth0
vethUVW
eth0 : 10.0.30.11
Worker
Container
eth0
vethABC
eth0 : 10.0.30.12
HOST network

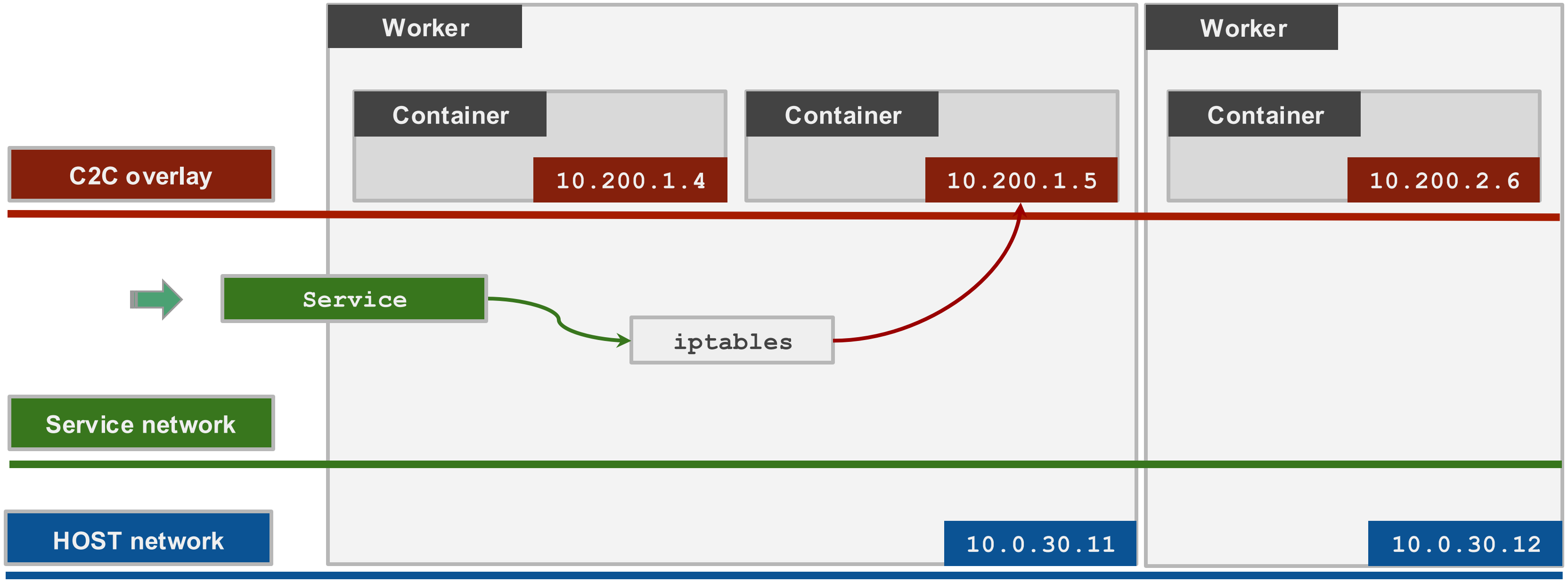
Networking


Worker
Container
eth0 : 10.200.1.4
Container
eth0 : 10.200.1.5
Worker
Container
eth0 : 10.200.2.6
C2C overlay
vethXYZ
vethUVW
vethABC
cni0
cni0
flannel0
flannel0
vxlan encap
vxlan decap
HOST network
eth0 : 10.0.30.11
eth0 : 10.0.30.12
ethernet frame
[outer] IP-in-IP header
[inner] IP datagram
src: host-a MAC
dst: $GATEWAY MAC
src: host-a IP
dst: host-b IP
IPinIP Header
src: pod-a IP
src: pod-b IP
tcp header
data
footer

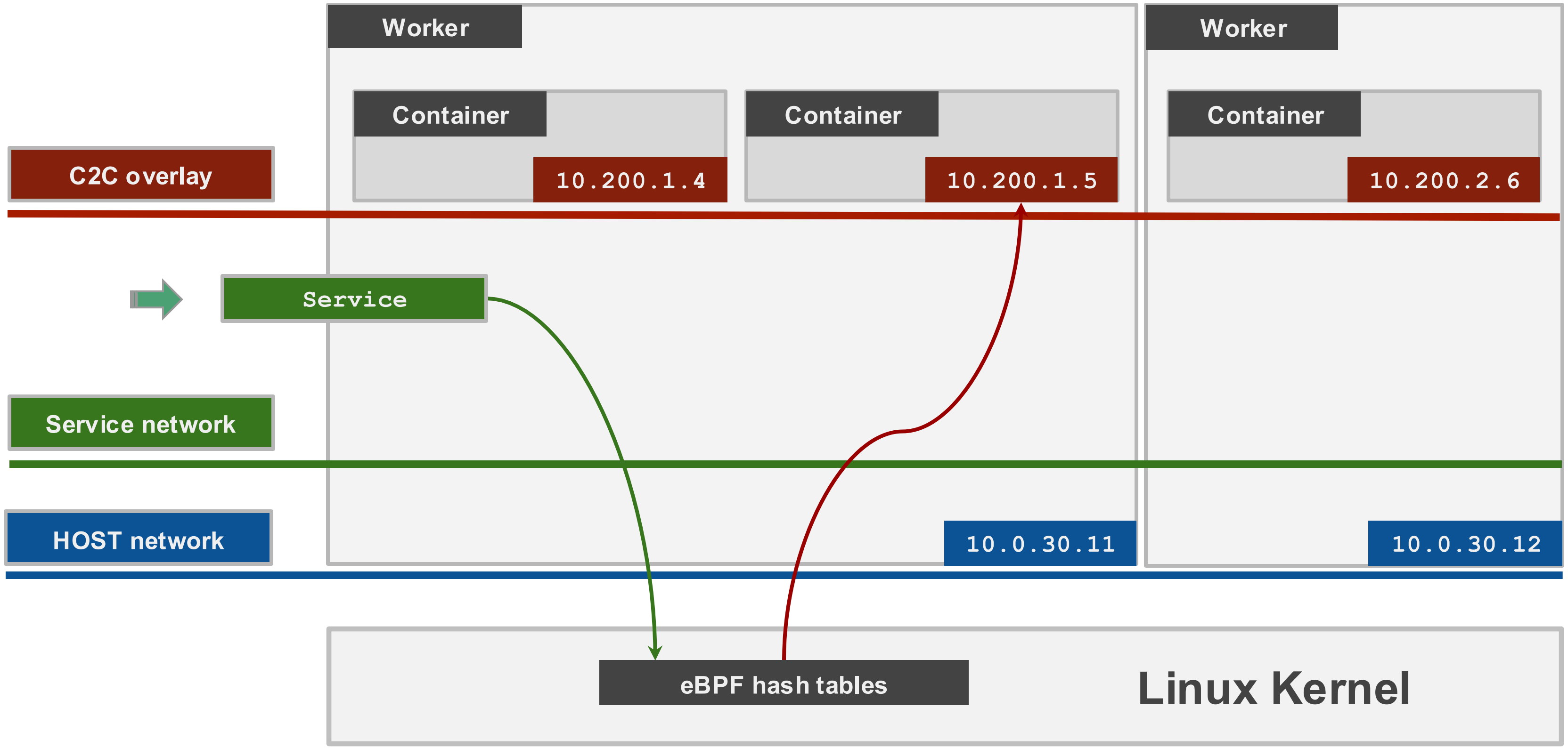
Networking


Worker
Container
10.200.1.4
Container
10.200.1.5
Worker
Container
10.200.2.6
C2C overlay
Service
kube-proxy
iptables
kube-proxy
iptables
Service network
HOST network
10.0.30.11
10.0.30.12

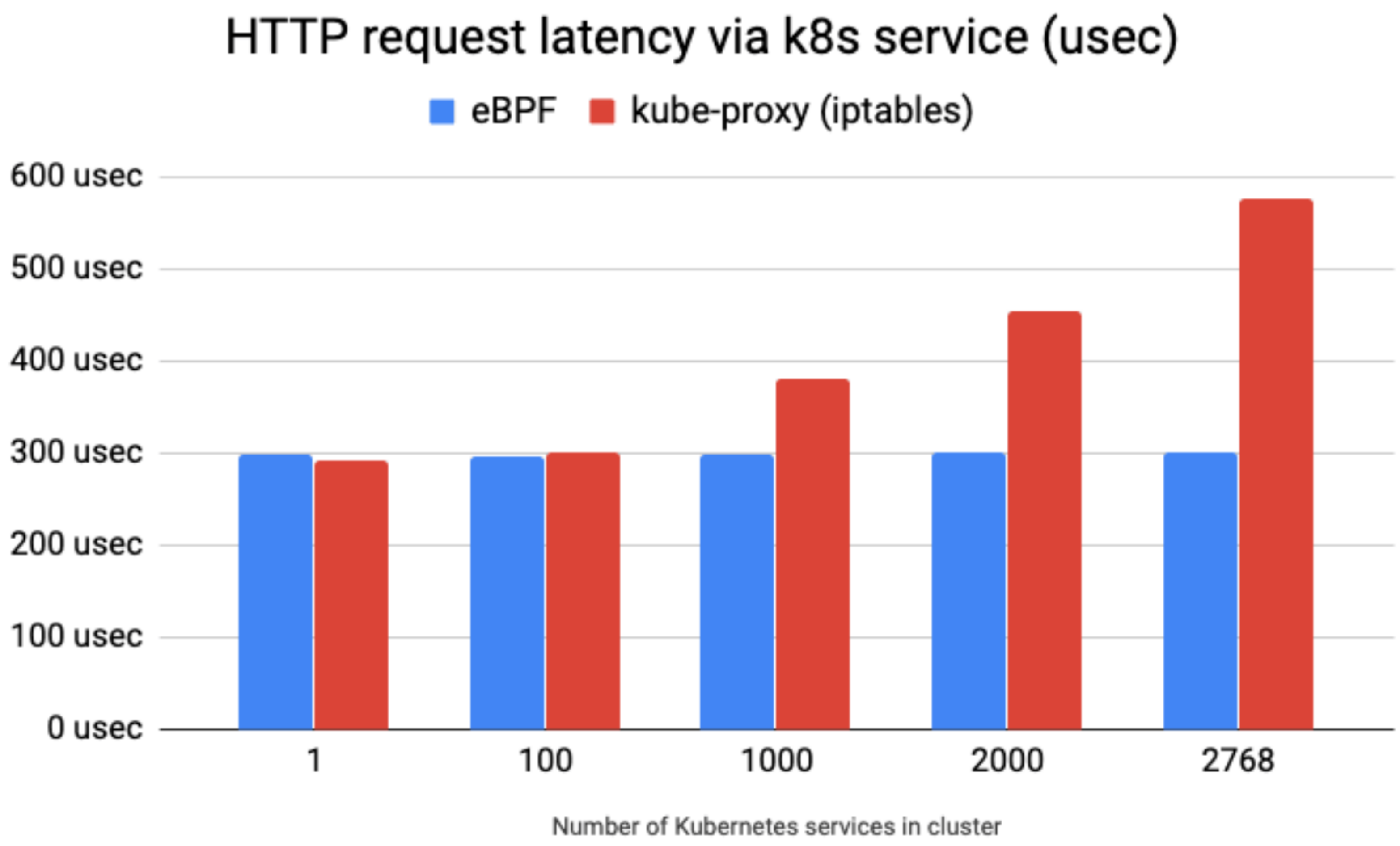
Networking


Worker
Worker
Container
Container
Container
C2C overlay
10.200.1.4
10.200.1.5
10.200.2.6
Service
iptables
Service network
HOST network
10.0.30.11
10.0.30.12

Networking



Worker
Container
10.200.1.4
Container
10.200.1.5
Worker
Container
10.200.2.6
C2C overlay
Service
Service network
HOST network
10.0.30.11
10.0.30.12
eBPF hash tables
Linux Kernel



HTTP request latency via k8s service (usec)
eBPF
kube-proxy (iptables)
600 usec
500 usec
400 usec
300 usec
200 usec
100 usec
0 usec
1
100
1000
2000
2768
Number of Kubernetes services in cluster

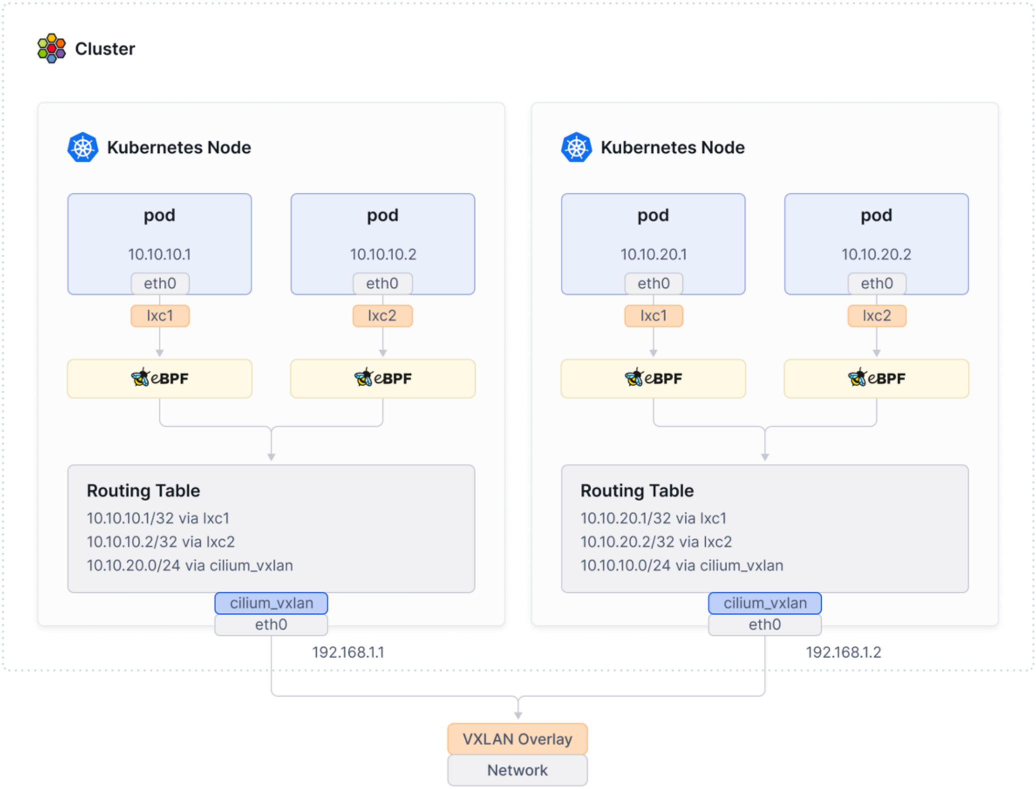
What is Cilium

- Cilium is an opensource solution, which is based on eBPF technology for cloud-native networking and security and was donated to the CNCF in 2021
- In 2023, Cilium is first on only Graduated CNCF project in cloud-native networking
- Adopted as the default CNI by most managed K8S services and distributions.
- In 2024, Cisco acquired its parent company, Isovalent.
- **Commitment to opensource**: Cisco has stated its commitment to continuing to support and nurture opensource communities for Cilium.

What is eBPF technology

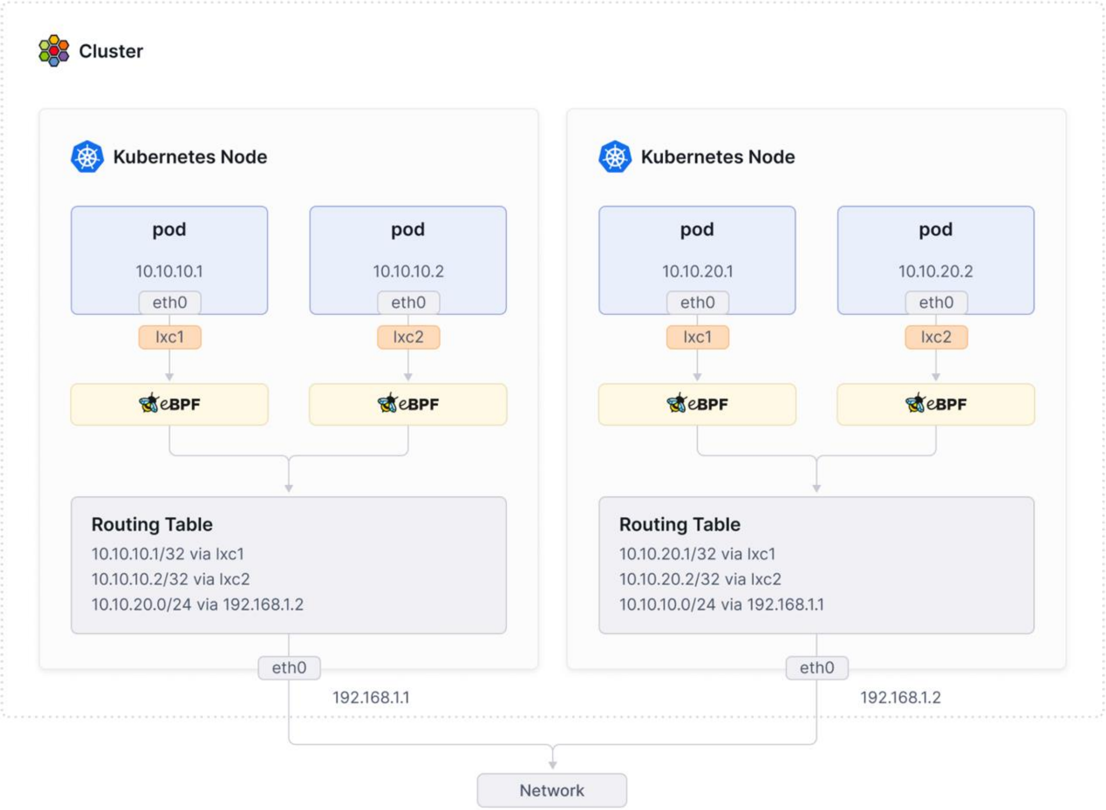
- **Kernel-level execution**: eBPF compiles programs into a bytecode that runs directly in kernel space, the core of the operating system.
- **Event-driven**: eBPF programs are triggered by specific events, such as system call, network packages, or function call in either the kernel or user space.
- **Dynamic extension**: Allow extension of functionality at runtime without needing to load a kernel module or change the kernel's source code.
- **Key use cases**: Networking (network filtering, lb, networkpolicy), Security (visibility into system calls, real-time threat detection), Observability (collect telemetry, tracing)

Cilium CNI – Encapsulating Routing Mode



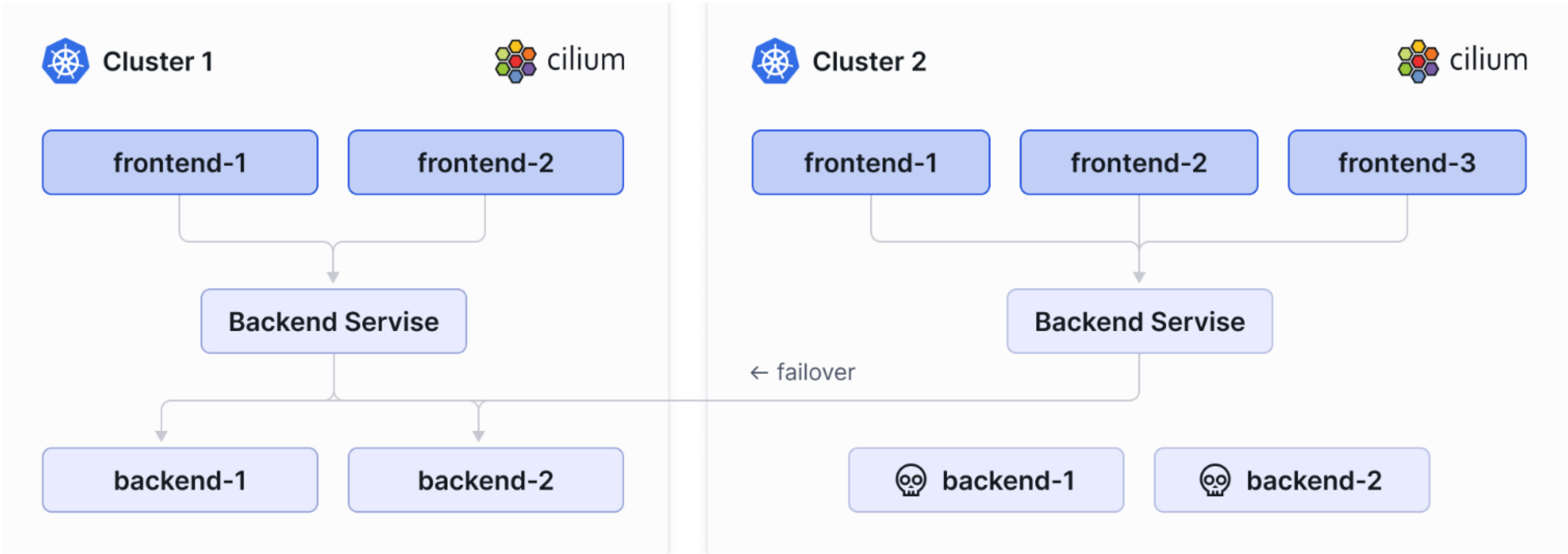
Encapsulation Routing Mode

Cilium CNI – Native Routing Mode

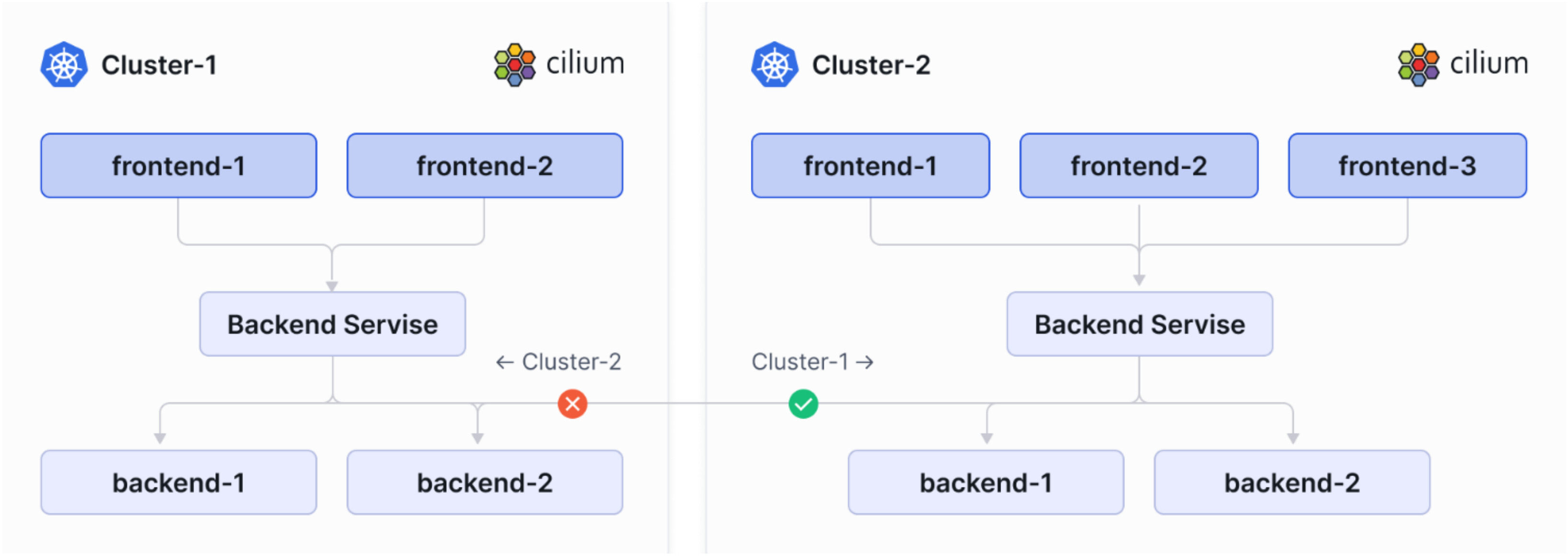


Native Routing Mode

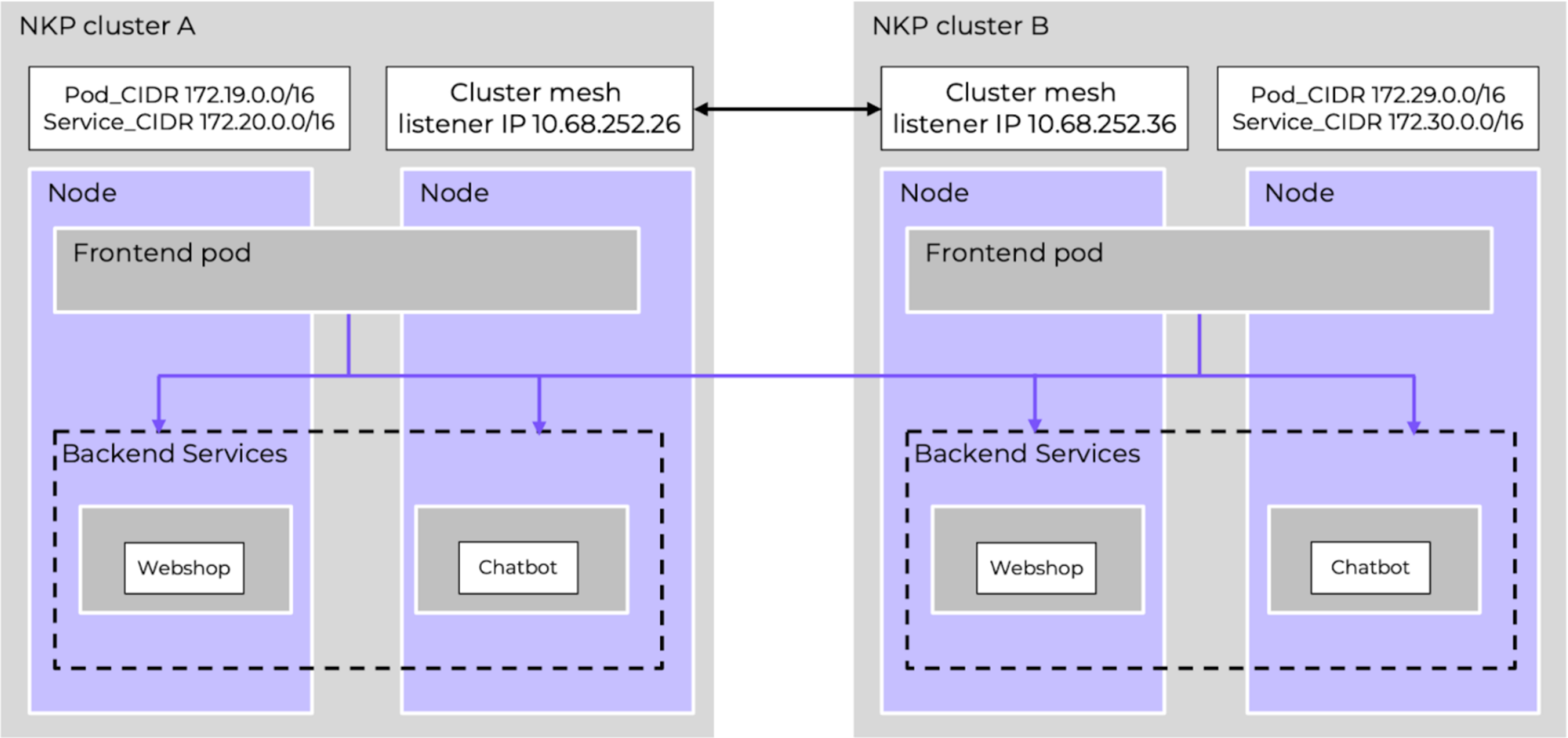
Cilium Cluster Mesh



Enforce network policy spanning multi cluster



Cilium Cluster Mesh Architecture



Cilium Cluster Mesh Architecture

Prerequisites

- CIDR ranges in all clusters and all nodes must be non-conflicting and unique IP addresses
- Nodes in all clusters must have IP connectivity between each other using the configured InternalIP for each node. This requirement is typically met by establishing peering or VPN tunnels between the networks of the nodes of each cluster
- The network between clusters must allow inter-cluster communication. The exact ports are documented in the Firewall Rules section
- Service Load Balancer IP range needs to have a minimum of 2 IP addresses (one for NKP, one for Cluster Mesh)

https://www.nutanix.dev/2025/09/17/how-to-enable-cilium-cluster-mesh-in-nutanix-kubernetes-platform/

Cilium Cluster Mesh Configmap

```yaml
apiVersion: cluster.x-k8s.io/v1beta1
kind: Cluster
metadata:
  name: <NAME>
  namespace: <WORKSPACE_NAMESPACE>
spec:
  topology:
    controlPlane:
      metadata:
        annotations:
          controlplane.cluster.x-k8s.io/skip-kube-proxy: ""
    variables:
      - name: clusterConfig
        value:
          addons:
            cni:
              provider: Cilium
              strategy: HelmAddon
              values:
                sourceRef:
                  name: cilium-clustermesh-clustera-config
                  kind: ConfigMap
```

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: cilium-clustermesh-clustera-config
  namespace: ${WORKSPACE_NAMESPACE}
data:
  values.yaml: |-
    # Cilium configuration values
    kubeProxyReplacement: true
    k8sServiceHost: "auto"
    tls:
      ca:
        cert: ${CA_CERT}
        key: ${CA_KEY}
    certgen:
      image:
        useDigest: false
    cluster:
      id: 1
      name: cluster-a
    clustermesh:
      apiserver:
        service:
          type: LoadBalancer
          loadBalancerIP: 10.68.252.26 # modify with your own IP
        tls:
          auto:
            enabled: true
            method: cronJob
            schedule: 0 0 1 */4 *
```

```
      config:
        clusters:
        - ips:
          - 10.68.252.36 # modify with your own IP
          name: cluster-b
          port: 2379
        enabled: true
      useAPIServer: true
    cni:
      chainingMode: portmap
      exclusive: false
    envoy:
      image:
        useDigest: false
    hubble:
      enabled: true
      relay:
        enabled: true
        image:
          useDigest: false
        priorityClassName: system-cluster-critical
        tls:
          server:
            enabled: true
            mtls: true
      tls:
        auto:
          certValidityDuration: 60
          enabled: true
          method: cronJob
          schedule: 0 0 1 * *
    image:
      useDigest: false
    ipam:
      mode: kubernetes
    operator:
      image:
        useDigest: false
    routingMode: tunnel
    socketLB:
      hostNamespaceOnly: true
    tunnelProtocol: vxlan
EOF
```

Cilium status

```
[nutanix@harbor dev1]$ KUBECONFIG=cluster-a.yaml cilium status --wait
    /¯¯\
 /¯¯\__/¯¯\    Cilium:             OK
 \__/¯¯\__/    Operator:           OK
 /¯¯\__/¯¯\    Envoy DaemonSet:    OK
 \__/¯¯\__/    Hubble Relay:       OK
    \__/       ClusterMesh:        OK

DaemonSet              cilium                   Desired: 2, Ready: 2/2, Available: 2/2
DaemonSet              cilium-envoy             Desired: 2, Ready: 2/2, Available: 2/2
Deployment             cilium-operator          Desired: 2, Ready: 2/2, Available: 2/2
Deployment             clustermesh-apiserver    Desired: 1, Ready: 1/1, Available: 1/1
Deployment             hubble-relay             Desired: 1, Ready: 1/1, Available: 1/1
Containers:            cilium                   Running: 2
                       cilium-envoy             Running: 2
                       cilium-operator          Running: 2
                       clustermesh-apiserver    Running: 1
                       hubble-relay             Running: 1
Cluster Pods:          18/18 managed by Cilium
Helm chart version:    1.17.4
Image versions         cilium                   quay.io/cilium/cilium:v1.17.4: 2
                       cilium-envoy             quay.io/cilium/cilium-envoy:v1.32.6-1746661844
                       cilium-operator          quay.io/cilium/operator-generic:v1.17.4: 2
                       clustermesh-apiserver    quay.io/cilium/clustermesh-apiserver:v1.17.4@s
                       hubble-relay             quay.io/cilium/hubble-relay:v1.17.4: 1
[nutanix@harbor dev1]$
```

```
[nutanix@harbor dev1]$ KUBECONFIG=cluster-b.yaml cilium status --wait
    /¯¯\
 /¯¯\__/¯¯\    Cilium:             OK
 \__/¯¯\__/    Operator:           OK
 /¯¯\__/¯¯\    Envoy DaemonSet:    OK
 \__/¯¯\__/    Hubble Relay:       OK
    \__/       ClusterMesh:        OK

DaemonSet              cilium                   Desired: 2, Ready: 2/2, Available: 2/2
DaemonSet              cilium-envoy             Desired: 2, Ready: 2/2, Available: 2/2
Deployment             cilium-operator          Desired: 2, Ready: 2/2, Available: 2/2
Deployment             clustermesh-apiserver    Desired: 1, Ready: 1/1, Available: 1/1
Deployment             hubble-relay             Desired: 1, Ready: 1/1, Available: 1/1
Containers:            cilium                   Running: 2
                       cilium-envoy             Running: 2
                       cilium-operator          Running: 2
                       clustermesh-apiserver    Running: 1
                       hubble-relay             Running: 1
Cluster Pods:          26/26 managed by Cilium
Helm chart version:    1.17.4
Image versions         cilium                   quay.io/cilium/cilium:v1.17.4: 2
                       cilium-envoy             quay.io/cilium/cilium-envoy:v1.32.6-1746661844-0f602c28cb2aa57b29078195049fb257d5b5246c: 2
                       cilium-operator          quay.io/cilium/operator-generic:v1.17.4: 2
                       clustermesh-apiserver    quay.io/cilium/clustermesh-apiserver:v1.17.4@sha256:0b72f3046cf36ff9b113d53cc61185e893edb5fe728a2c9e561c1083f806453d: 3
                       hubble-relay             quay.io/cilium/hubble-relay:v1.17.4: 1
[nutanix@harbor dev1]$
```

Cluster Mesh status

```
[nutanix@harbor dev1]$ KUBECONFIG=cluster-a.yaml cilium clustermesh status --wait
✅ Service "clustermesh-apiserver" of type "LoadBalancer" found
✅ Cluster access information is available:
  - 10.38.19.210:2379
✅ Deployment clustermesh-apiserver is ready
ℹ️  KVStoreMesh is enabled

✅ All 2 nodes are connected to all clusters [min:1 / avg:1.0 / max:1]
✅ All 1 KVStoreMesh replicas are connected to all clusters [min:1 / avg:1.0 / max:1]

🔌 Cluster Connections:
  - cluster-b: 2/2 configured, 2/2 connected - KVSt

[nutanix@harbor dev1]$
```

```
[nutanix@harbor dev1]$ KUBECONFIG=cluster-b.yaml cilium clustermesh status --wait
✅ Service "clustermesh-apiserver" of type "LoadBalancer" found
✅ Cluster access information is available:
  - 10.38.19.213:2379
✅ Deployment clustermesh-apiserver is ready
ℹ️  KVStoreMesh is enabled

✅ All 2 nodes are connected to all clusters [min:1 / avg:1.0 / max:1]
✅ All 1 KVStoreMesh replicas are connected to all clusters [min:1 / avg:1.0 / max:1]

🔌 Cluster Connections:
  - cluster-a: 2/2 configured, 2/2 connected - KVStoreMesh: 1/1 configured, 1/1 connected

[nutanix@harbor dev1]$
```

Cluster Mesh – Global Service (cluster A)

```yaml
# shared-service.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: shared-service
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  namespace: shared-service
spec:
  selector:
    matchLabels:
      app: nginx
  replicas: 2
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: shared-service
  namespace: shared-service
  annotations:
    service.cilium.io/global: "true" # Crucial for Cluster Mesh service d
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP
```

Cluster Mesh – Global service (cluster B)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: shared-service
  namespace: shared-service
  annotations:
    service.cilium.io/global: "true" # Crucial for Cluster Mesh service d
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP
```

Cluster Mesh – Test Pod (cluster B)

```yaml
# client-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: client-pod
  namespace: shared-service
spec:
  containers:
  - name: client
    image: curlimages/curl:latest
    command: ["tail", "-f", "/dev/null"] # Keep the pod running
```

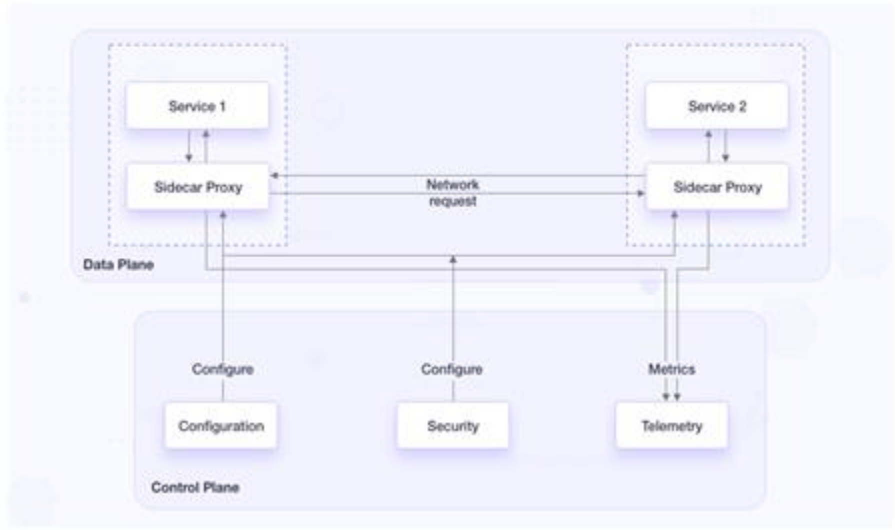
Cluster Mesh – Test Pod (cluster B)

```
[nutanix@harbor dev1]$ kubectl -n shared-service exec client-pod -- curl shared-service.shared-service
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   615 100   615    0     0 244338      0 --:--:-- --:--:-- --:--:-- 307500
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
[nutanix@harbor dev1]$
```

One More Thing...

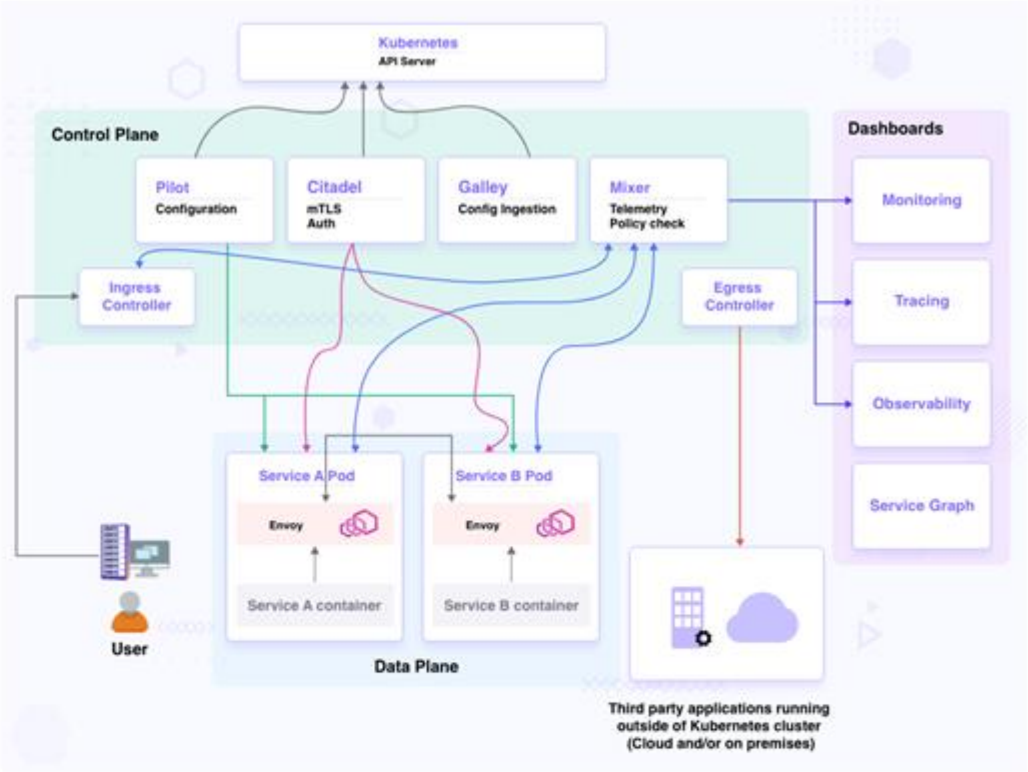
Service Mesh

- A dedicated infrastructure layer for managing service-to-service communication.
- Data Plane: The layer responsible for handling actual application traffic (Proxies that handle traffic)
- Control plane: It configures and manages the data plane. Management brain that tells proxies what to do.



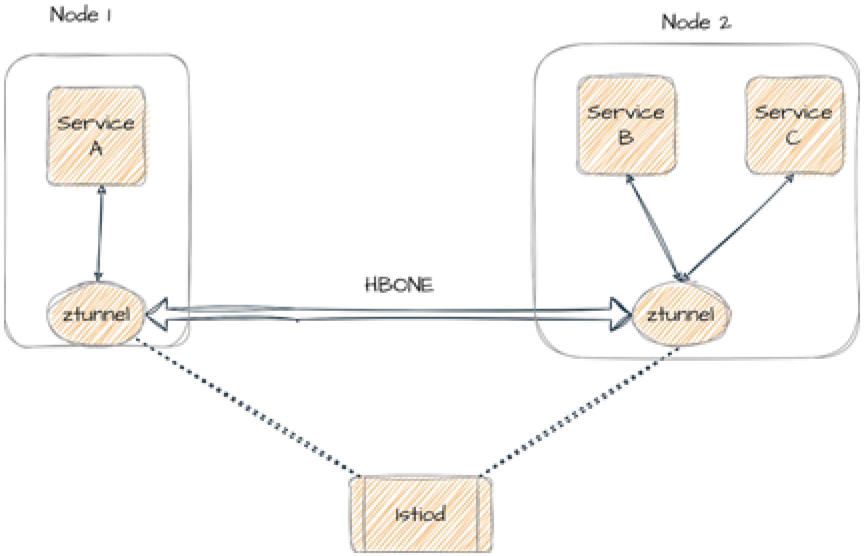
How it works (Sidecar)

- Each pod gets a proxy container e.g. Envoy.
- All inbound/outbound traffic goes through the proxy
- Service Mesh control plane **istiod** configures proxies
- Pilot + Citadel + Galley merged into istiod.



How it works (Ambient Mode)

- No sidecars no maintenance overhead
- Traffic intercepted by proxy runs once per node DaemonSet, L4 secure overlay
- ztunnel per-node proxy that transparently intercepts pod traffic
- It tunnels traffic using HBONE (HTTP Based Overlay Network). Multiple TCP streams multiplex over a single mTLS connection.



When to use

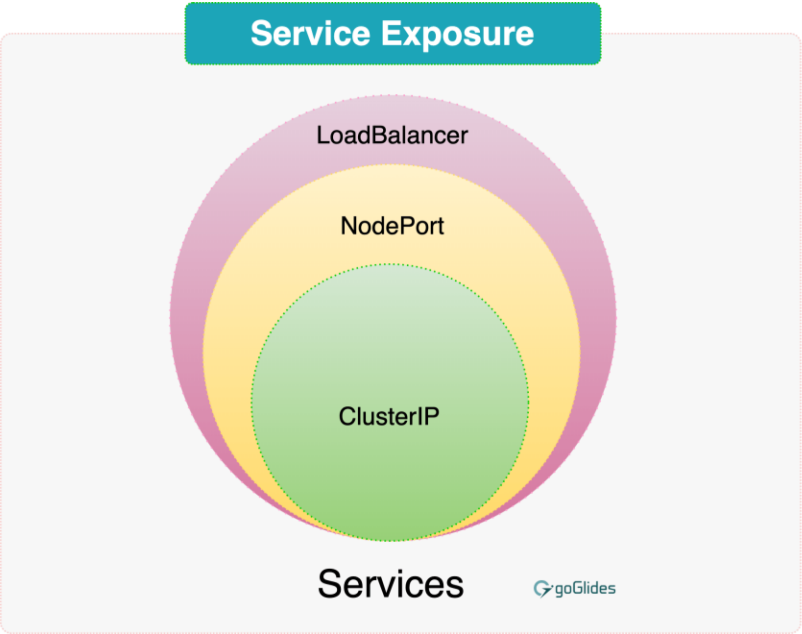
- Many microservices needing secure, reliable communication
- Want observability without adding code
- Need fine-grained traffic control
- expect service count to grow fast
- run multi-cluster / multi-cloud
- want security, routing, and monitoring without touching app code
- want easier certificate rotation

When not to use

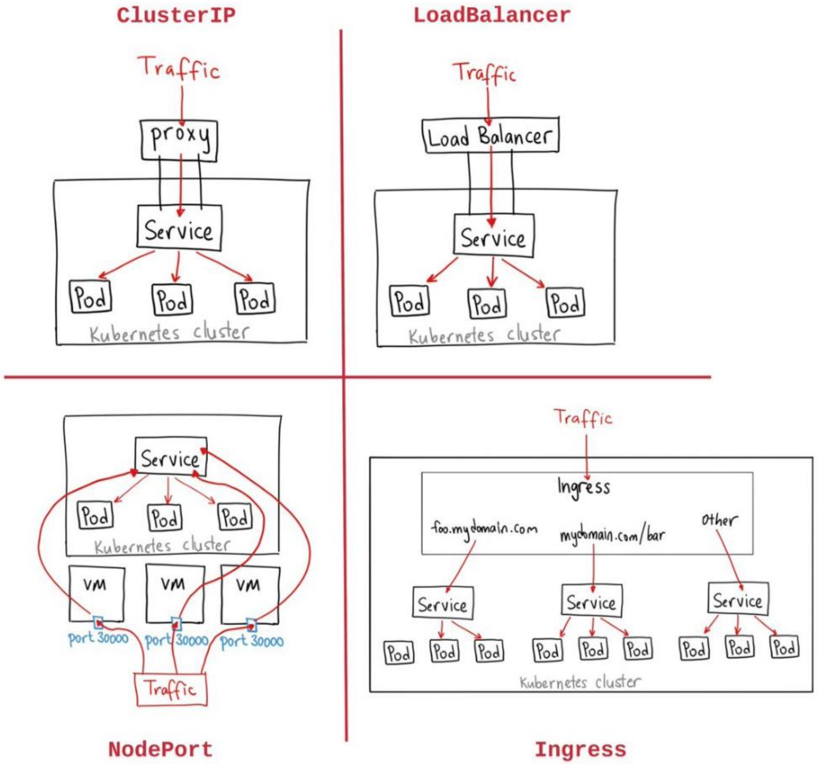
- Have a small number of services, few services (say <20–30)
- Communication is mostly internal & trusted, stays inside one trusted cluster
- Don’t urgently need centralized traffic policies or observability
- Performance overhead is a major concern
- Fine maintaining mTLS in code & cert rotation manually

One More Thing...

Gateway API



Service Exposure
LoadBalancer
NodePort
ClusterIP
Services
goGlides



ClusterIP
Traffic
proxy
Service
Pod
Pod
Pod
Kubernetes cluster
LoadBalancer
Traffic
Load Balancer
Service
Pod
Pod
Pod
Kubernetes cluster
Service
Pod
Pod
Pod
Kubernetes cluster
VM
VM
VM
port 30000
port 30000
port 30000
Traffic
NodePort
Traffic
Ingress
foo.mydomain.com
mydomain.com/bar
other
Service
Service
Service
Pod
Pod
Pod
Pod
Pod
Pod
Pod
Pod
Pod
Kubernetes cluster
Ingress

Ingress is lagacy

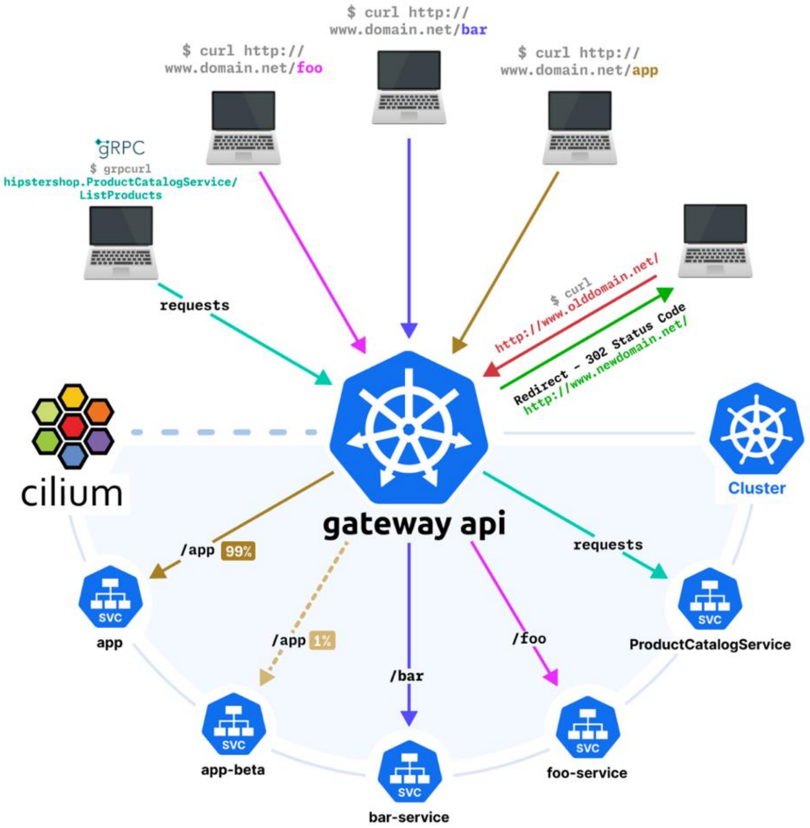
```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: basic-ingress
  namespace: default
spec:
  ingressClassName: cilium
  rules:
  - http:
      paths:
      - backend:
          service:
            name: details
            port:
              number: 9080
        path: /details
        pathType: Prefix
      - backend:
          service:
            name: productpage
            port:
              number: 9080
        path: /
        pathType: Prefix
```

Gateway API

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: my-gateway
spec:
  gatewayClassName: cilium
  listeners:
  - protocol: HTTP
    port: 80
    name: web-gw
    allowedRoutes:
      namespaces:
        from: Same
```

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: http-app-1
spec:
  parentRefs:
  - name: my-gateway
    namespace: default
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /details
    backendRefs:
    - name: details
      port: 9080
  - matches:
    - headers:
      - type: Exact
        name: magic
        value: foo
      queryParams:
      - type: Exact
        name: great
        value: example
      path:
        type: PathPrefix
        value: /
      method: GET
    backendRefs:
    - name: productpage
      port: 9080
```

Gateway API Use Cases



Gateway API Use Cases

NUTANIX

“Network Mesh Implementation with Cilium”

ร่วมทำแบบสอบถามเพื่อรับของที่ระลึกจากนูทานิคซ์

กระเป๋าอเนกประสงค์ / เป๋าสายไฟ

Thank you